networks. The machine where the Docker server runs is called the *Docker host*. Each machine where you want to run containers should be a Docker host, so it should have a Docker daemon running. The portability of containers is made possible by the daemon process itself.

The Docker daemon exposes an API you can use to send instructions, such as to run a container or create a volume. The *Docker client* talks to the daemon through that API. The client is command-line based and can be used to interact with the Docker daemon either through scripting (for example, Docker Compose) or through the Docker CLI directly.

Besides the client and server components characterizing the Docker Engine, another essential element of the platform is a *container registry*, which has a similar function to a Maven repository. While Maven repositories are used to host and distribute Java libraries, container registries do the same for container images, and they follow the OCI Distribution Specification. We distinguish between public and private registries. The Docker company provides a public registry called Docker Hub (https://hub.docker.com), which is configured by default with your local Docker installation and hosts images for many popular open source projects, like Ubuntu, PostgreSQL, and OpenJDK.

Based on the architecture description included in the Docker documentation (https://docs.docker.com), figure 2.7 shows how the Docker client, Docker server, and container registry interact.